Ground Control — Demo Run of Show

2 intro slides + the FULL ./demo.sh (8 beats) · target ~2:58 · hard cap 3:00 · WHAT = what runs · SAY = read aloud · DO = keystrokes · NOTES = extra detail for explaining the use case

Pacing is tight. Move briskly and press Enter promptly. **Trim levers if long:** drop the slides, or skip Beat 7 (JSON) and show only Beat 7b (the bundle).

PART 1 — SLIDES (0:00–0:24) present intro-slides.pptx, keep brief

Slide 1 — The Problem (0:00–0:12)

SAY "A thousand-GPU training run slows down — or dies. Finding out why is hours of hand-digging through logs and counters across many nodes while the GPUs sit idle — and it looks the same whether it's bad hardware or one wrong setting."

NOTES Set the stakes: GPU time is the most expensive thing we sell, so a stalled run is money burning. The hard part isn't fixing it — it's *finding* it, by hand, across many machines. Hardware fault vs. a config mistake look identical from the outside.

Slide 2 — The Solution (0:12–0:24)

SAY "Ground Control: one command names the root cause and the fix in 30 seconds — for ops, customers, and support. Let me show you." → switch to the terminal.

NOTES The differentiator is the **verdict engine**: it doesn't just list red/green checks, it *correlates* them into one named cause with the action. Same tool, three audiences. Deterministic, runs on real hardware — no AI cost to run. Then pivot to the live terminal.

PART 2 — LIVE DEMO (0:24–~2:58) run: ./demo.sh

Intro banner (0:24–0:28)

WHAT starts the demo runner; prints the one-line problem banner.

SAY "Here it is, live."

DO `press Enter.`

NOTES Don't dwell — the slides already made the pitch. One line and move into the catalog.

Beat 1 — The catalog (0:28–0:40) audit -l

WHAT lists all 51 checks (groups A/B/C/G + NCCL) and the use-case bundles.

SAY “51 checks across the server, the network, and the GPUs. It auto-detects the hardware, and bundles the right checks per symptom.”

DO press Enter.

NOTES Point at the right-hand column: **native** = runs here; **NA** = auto-skipped because this GPU lacks that feature (NVLink/IMEX are GB200/NVL multi-GPU features, not RTX PRO 6000); **active (red)** = traffic-generating tests, off by default so they never disturb a live cluster. The bundles at the bottom (preflight, nccl_slow, gpu_fault…) are named recipes — the menu and every demo just call these. Takeaway: host + network + GPU in one tool that adapts to whatever hardware it lands on.

Beat 2 — A healthy node (0:40–0:54) audit -u preflight --fixture

fixtures/cw0bec_blackwell_healthy

WHAT runs the preflight bundle against a REAL capture from a live CoreWeave

Blackwell node.

SAY “Real data from a live CoreWeave Blackwell node — eight GPUs, nine network cards. All green, exit zero: you can gate CI and stop a bad node before it ever gets a job.”

DO press Enter.

NOTES Call out “17 pass, 0 fail” and **exit 0** — the exit code is the automation hook: a CI gate or scheduler runs this and refuses to launch on a node that isn’t green. Point at A6 (real NUMA topology — which GPUs sit electrically close to which NIC) and A5 (real kernel/driver/firmware versions): proof this is a live node, not mocked. This is the known-good baseline, so the failures next are obvious by contrast.

Beat 3 — A failing GPU (0:54–1:10) audit -u gpu_fault --fixture fixtures/synth_gpu_fault

WHAT runs the GPU-fault bundle on a failing-GPU capture.

SAY “First, a failing GPU. Three separate signs on the same GPU — driver errors, failing memory, and ninety-one degrees — collapse into one verdict: fence the node off and replace the card.”

DO press Enter.

NOTES Three red lines: G5 = Xid (NVIDIA driver/hardware error codes), G3 = uncorrectable ECC

(failing GPU memory), G7 = 91°C. Individually, an engineer chases each one. The point: the verdict at the top fuses all three *because they're on the same gpu:3*, and gives the action. "Cordon" = tell the scheduler to stop placing work here; "RMA" = get the card replaced. This is correlation on a shared scope — the core idea in miniature.

Beat 4 — NCCL on TCP · THE HEADLINE (1:10–1:42 · slow down) audit -u nccl_slow --

fixture fixtures/nccl_tcp_fallback

WHAT runs the NCCL-slow bundle against real NCCL logs where NCCL fell back to

TCP.

SAY "Now the big one. GPUs sync data during training through a library called NCCL. Normally it uses RDMA — the fast path where network cards talk directly and skip the CPU. One wrong setting silently dropped it to ordinary TCP — about a hundred times slower — looking just like 'the cluster is slow.' The verdict names the cause AND the exact fix."

DO press Enter.

NOTES The money moment — slow down. The insidious part: **nothing is broken** — every hardware check (B12/B13/B14) is green, the fabric is healthy. It's a *config* regression (wrong NCCL_IB_HCA) that just looks like generic slowness, so teams can burn days. The verdict names it and the exact knob to check. If asked, tie it to the real GB300 incident this month — this exact class of failure hit a live customer and took a multi-day war-room; Ground Control turns that into a one-line verdict. This is the business case.

Beat 5 — A dead RDMA port (1:42–1:58) audit -u preflight --fixture

fixtures/synth_fabric_fault

WHAT runs preflight on a fabric-fault capture (a down port, a port throwing errors, a

congested port).

SAY "A dead network port. One port down, another throwing errors, a third congested — the verdict pins the dead rail and the action: check the cable, drain the node."

DO press Enter.

NOTES Three network symptoms at once: B13 a port DOWN (mlx5_5), B14 rising error counters on mlx5_4 (symbol errors = a flaky cable or optic), B11 congestion on mlx5_6. The verdict pins the dead one (mlx5_5) and the action. A "rail" is one of a node's parallel NIC-to-switch paths — lose one and

multi-node bandwidth craters. Shows the tool correlates *network* faults, not just GPU.

Beat 6 — Customer profile (1:58–2:14) audit --profile customer -u nccl_slow --fixture .../nccl_tcp_fallback

WHAT the same NCCL run, through the customer profile.

SAY “Same engine, three audiences. In customer mode it’s read-only, internal names are hidden, and every failure carries a plain-English fix and a ‘contact support’ line.”

DO press Enter.

NOTES Explicitly compare to Beat 4: the node name is now “node:<host>” (redacted), it says read-only, and each failure gains the remediation arrows + “Contact CoreWeave support.” Same engine, same finding — only safety, redaction, and wording changed. This is how a customer who isn’t a GPU expert gets the same answer without seeing internal details.

Beat 7 — Support · JSON (2:14–2:26 · TRIM LEVER: skippable) audit --profile support ... --json

WHAT emits structured JSON — verdict, severity, scope, supporting checks.

SAY “For support and automation, the same run emits structured JSON — the verdict, how severe, where, and the checks behind it.”

DO press Enter.

NOTES Point at the fields: exit_code (2 = failures, CI-friendly), the summary counts, and the verdicts array with id / severity / scope / supporting. This is what you feed into dashboards, ticketing, or automation. If you’re short on time, skip this and go straight to the bundle (7b).

Beat 7b — Support · the bundle (2:26–2:44) audit --profile support ... --bundle

WHAT writes a redacted tarball (result.json + raw evidence).

SAY “And a redacted bundle — the result plus the raw evidence — to attach straight to a support ticket. Self-serve, into a clean handoff.”

DO press Enter.

NOTES It writes audit-bundle-*.tar.gz. Inside: result.json + the raw evidence files, all redacted. This is the single artifact a customer attaches to a ticket, so support has everything up front — no “can you run X and send the output” back-and-forth. Closes the self-serve → support loop.

Close (2:44–2:58)

WHAT the summary banner.

SAY “Faster runs and less idle spend for customers; fewer escalations and higher fleet use for CoreWeave. And the foundation to predict these failures before they strike. Ground Control.”

NOTES Land the three audiences and the moonshot in one breath: this is the walking skeleton; the future is predicting these failures before they strike. End on the name.

Keep under 3:00. Rubric — Speed: Slide 1 & Beat 4 · Business: Slides, Beats 6–7b, Close · Technical: Beats 1–5 · Presentation: the whole Problem → Solution → Prototype → Impact arc.